

Experiment – 3

Date: 19/09/2026

Aim of the Experiment:

Working with Digital and analog Input and Implementation of LED circuits using push buttons.

Objectives:

- 1) To understand the concept of pull-up and pull-down resistor and interfacing a push button to control an LED.
- 2) To utilize the push button as a toggle switch.
- 3) To control the variation in brightness in LED by using two push buttons (increment in the brightness through one push button and decrement the brightness through other push button).
- 4) Implementation of a mini piano using push buttons and Buzzer.

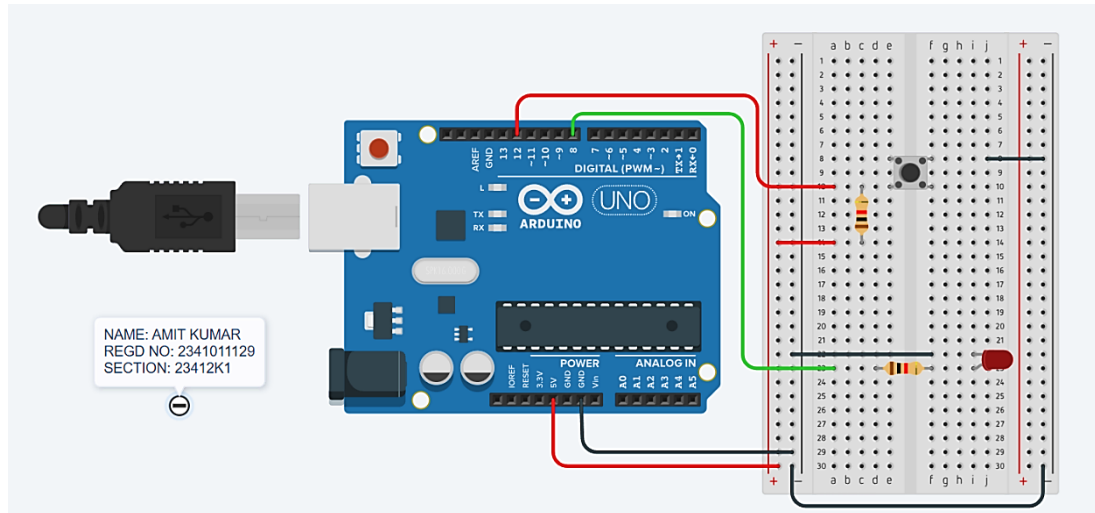
Components/Equipment Required:

Sl No.	Name of the Component/Equipement	Specification	Quantity
1	Arduino UNO R3	16MHz	1
2	Arduino UNO cable	USB Type A to B	1
3	Resistors (carbon type)	¼ watt (220Ω)	1
		¼ watt (10kΩ)	2
		¼ watt (1MΩ)	1
4	LED	Red	1
5	Push Buttons		7
6	Buzzer	Piezo	1
7	Breadboard	840 Tie points	1
8	Jumper Wire	-----	As per requirement

Objective 1

To understand the concept of pull-up and pull-down resistor and interfacing a push button to control an LED.

Circuit/Schematic Diagram:



Code:

```
int LEDPin=8;
int buttonPin=12;
int buttonRead;
int dt=100;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  pinMode(LEDPin, OUTPUT);
  pinMode(buttonPin, INPUT);
}

void loop() {
  // put your main code here, to run repeatedly:
  buttonRead=digitalRead(buttonPin);
  Serial.println(buttonRead);
  delay(dt);

  if(buttonRead==1){
    digitalWrite(LEDPin, LOW);
  }

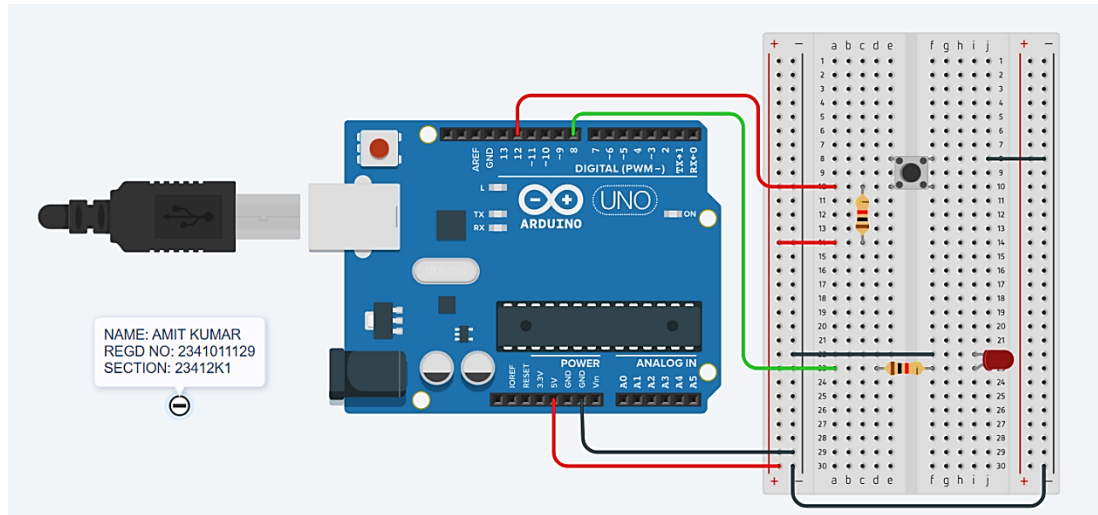
  if(buttonRead==0){
    digitalWrite(LEDPin, HIGH);
  }
}
```

Observation: The Arduino Uno is interfaced with a push button and LED. When the button is pressed, the LED turns ON, and when released, the LED turns OFF. The button state is also displayed on the Serial Monitor.

Objective 2

To utilize the push button as a toggle switch

Circuit/Schematic Diagram:



Code:

```
int pushButtonPin=12;
int redLEDpin=8;
int delayTime=250;
int pushButtonRead;
int LEDState=0;
int buttonOld=1;
int buttonNew;

void setup() {
  Serial.begin(9600);
  pinMode(pushButtonPin, INPUT);
  pinMode(redLEDpin, OUTPUT);
}

void loop() {

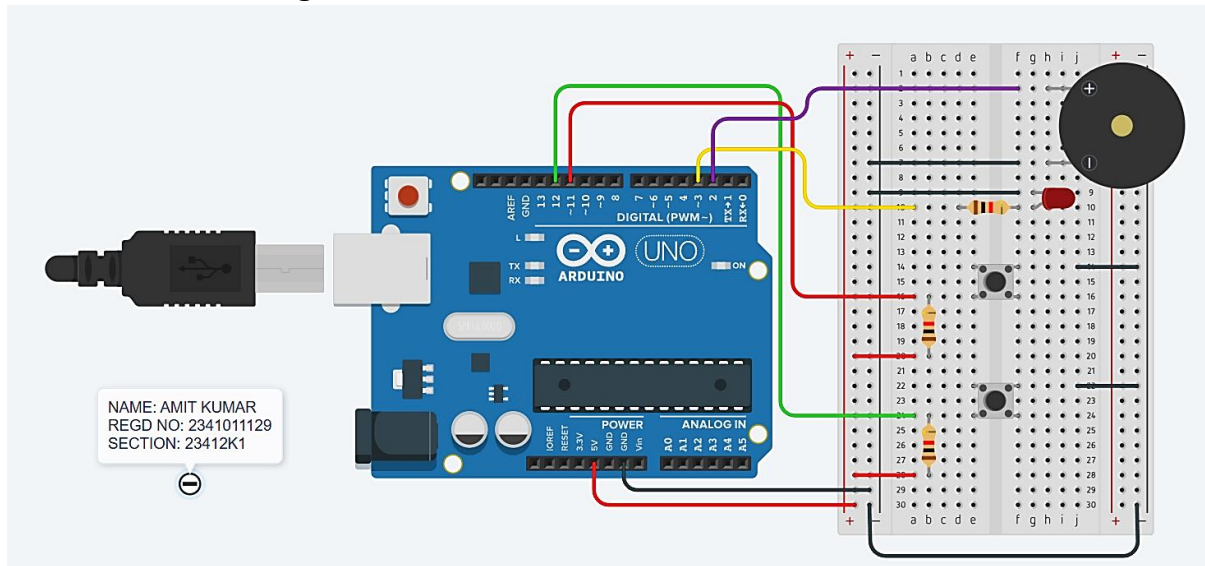
  buttonNew=digitalRead(pushButtonPin);
  if(buttonOld==0 && buttonNew==1)
  {
    if(LEDState==0)
    {
      digitalWrite(redLEDpin, HIGH);
      LEDState=1;
    }
    else{
      digitalWrite(redLEDpin, LOW);
      LEDState=0;
    }
  }
  buttonOld=buttonNew;
  delay(delayTime);
}
```

Observation: The circuit uses an Arduino Uno, push-button, resistor, and LED. When the push-button is pressed, the Arduino detects the input and turns the LED ON; when released, the LED remains OFF. The program also uses a short delay for button debouncing.

Objective 3

To control the variation in brightness in LED by using two push buttons (increment in the brightness through one push button and decrement the brightness through other push button).

Circuit/Schematic Diagram:



Code:

```
int pushButtonPin1=12;
int pushButtonPin2=11;
int redLEDpin=3;
int buzzPin=2;
int buttonRead1;
int buttonRead2;
int LEDBright=0;
int delayTime=500;

void setup() {
  pinMode(pushButtonPin1, INPUT);
  pinMode(pushButtonPin2, INPUT);
  pinMode(redLEDpin, OUTPUT);
  pinMode(buzzPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  buttonRead1=digitalRead(pushButtonPin1);
  buttonRead2=digitalRead(pushButtonPin2);

  Serial.print("Button 1 = ");
  Serial.print(buttonRead1);
  Serial.print(" , ");

  Serial.print("Button 2 = ");
  Serial.println(buttonRead2);

  delay(delayTime);

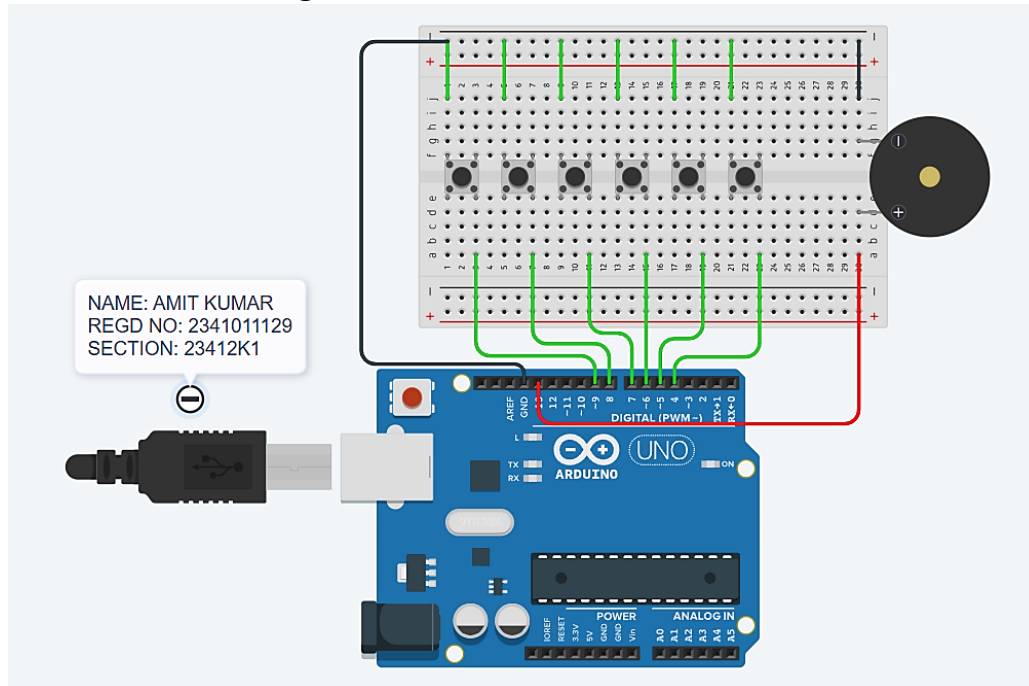
  if(buttonRead1==0){
    LEDBright=LEDBright+5;
  }
  if(buttonRead2==0){
    LEDBright=LEDBright-5;
  }
  Serial.println(LEDBright);
  if(LEDBright>255){
    LEDBright=255;
    digitalWrite(buzzPin,HIGH);
    delay(delayTime);
    digitalWrite(buzzPin,LOW);
    Serial.println("Buzz is ON");
  }
  if(LEDBright<0){
    LEDBright=0;
    digitalWrite(buzzPin,HIGH);
    delay(delayTime);
    digitalWrite(buzzPin,LOW);
    Serial.println("Buzz is OFF");
  }
  analogWrite(redLEDpin,LEDBright);
}
```

Observation: The circuit uses an Arduino Uno to control an LED and buzzer through two push buttons. Pressing the buttons changes the LED brightness, and the buzzer turns ON at maximum brightness and OFF at minimum brightness. The serial monitor displays the LED brightness and buzzer status.

Objective 4:

Implementation of a mini piano using push buttons and Buzzer.

Circuit/Schematic Diagram:



Code:

```
int cPin=9, dPin=8, ePin=7, fPin=6,
gPin=5, aPin=4;
int duration = 200;
void setup() {
  pinMode(13, OUTPUT);
  pinMode(cPin, INPUT_PULLUP);
  pinMode(dPin, INPUT_PULLUP);
  pinMode(ePin, INPUT_PULLUP);
  pinMode(fPin, INPUT_PULLUP);
  pinMode(gPin, INPUT_PULLUP);
  pinMode(aPin, INPUT_PULLUP);
}
void loop() {
  if(digitalRead(cPin)==LOW) {
    tone(13,261,duration); // C
    delay(600);
  }
  if(digitalRead(dPin)==LOW) {
    tone(13,293,duration); // D
    delay(600);
  }
  if(digitalRead(ePin)==LOW) {
    tone(13,329,duration); // E
    delay(600);
  }
  if(digitalRead(fPin)==LOW) {
    tone(13,349,duration); // F
    delay(600);
  }
  if(digitalRead(gPin)==LOW) {
    tone(13,392,duration); // G
    delay(600);
  }
  if(digitalRead(aPin)==LOW) {
    tone(13,440,duration); // A
    delay(600);
  }
}
```

Conclusion: The experiment was successfully performed using the Arduino UNO to interface push buttons, LED, and buzzer. The concepts of digital input, pull-up/pull-down resistors, push-button toggle operation, and PWM-based LED brightness control were understood and implemented. The two push buttons were used to increase and decrease LED brightness, while the buzzer indicated the maximum and minimum brightness conditions. Finally, multiple push buttons were interfaced with a buzzer to implement a mini piano, demonstrating how different button inputs can generate different tones.

Precautions:

1. Check all Arduino and breadboard connections before powering the circuit.
2. Use a $220\ \Omega$ resistor with the LED to limit current and protect the LED.
3. Ensure the push buttons and their pull-up/pull-down resistors are connected correctly.
4. Do not short-circuit the Arduino's 5V and GND supply rails.
5. Connect the buzzer and LED to the correct Arduino pins as specified in the program.
6. Use proper button debouncing to avoid multiple unwanted button detections; the experiment uses a short delay for this purpose.
7. Verify the polarity of the LED before connecting it.
8. Upload the correct program and select the appropriate Arduino UNO board and COM/port.

Post Experiment Questionnaire:

1. Differentiate between an active and a passive buzzer.

Solⁿ-

Active Buzzer	Passive Buzzer
Has an internal oscillator.	Does not have an internal oscillator.
Produces sound when DC voltage is applied.	Requires an alternating/PWM signal to produce sound.
Frequency is generally fixed.	Frequency can be controlled by the input signal.
Easier to use for simple ON/OFF sound.	Suitable for generating different musical tones.

2. To read the data from a push button, which Arduino inbuilt function is used?

Solⁿ- The Arduino function used to read a push-button's digital state is:- `digitalRead(pin)`.
Eg - `buttonState = digitalRead(buttonPin);`

3. Can a push button be used as a toggle switch? Justify your answer.

Solⁿ- Yes, A push button can be used as a toggle switch by storing its previous state and changing the output state only when a new button press is detected.

For example:

First press → LED ON

Second press → LED OFF

Third press → LED ON

4. Specify the range of values for Arduino in reading and writing a signal.

Solⁿ-

Operation	Function	Value Range
Digital Reading	digitalRead()	LOW / HIGH
Digital Writing	digitalWrite()	LOW / HIGH
Analog Reading	analogRead()	0–1023
PWM Writing	analogWrite()	0–255

5. How to install a library in an Arduino sketch (for zip and unzip format).

Solⁿ- For ZIP format:

1. Open Arduino IDE.
2. Go to Sketch → Include Library → Add .ZIP Library.
3. Select the required .zip library file.
4. Click Open; the library will be installed.

For Unzipped format:

1. Extract the library folder.
2. Copy the library folder into the Arduino libraries directory.
3. Restart Arduino IDE.
4. Include the library using Sketch → Include Library or #include <LibraryName.h>.

6. Differentiate between “const” and “#define”.

Solⁿ-

const	#define
A C/C++ language feature.	A preprocessor directive.
Defines a typed constant variable.	Performs text/macro substitution before compilation.
Follows normal C/C++ scope rules.	Does not create a variable or have normal type checking.
Example: const int pin = 13;	Example: #define PIN 13

Name of the Student: Amit Kumar
Registration No: 2341011129
Branch, Section: CSE, 23412K1